

1. Target Sum:-

nums = [1, 1, 1, 1, 1]

target = 3

o/p = 5

+ , - to each & make target = 3.

Subset sum problem:-

total sum = S

P = sum of numbers you assign a +

N = sum of numbers you assign a -

$$\therefore P - N = \text{target} \rightarrow \textcircled{1}$$

all numbers are used exactly once either in P or N.

$$\therefore P + N = \text{sum} \rightarrow \textcircled{2}$$

$$\therefore 2P = \text{target} + \text{sum}$$

$$P = (\text{target} + \text{sum}) // 2$$

$\therefore$ Instead of assigning + / - we just

Find subsets whose sum = P

$\therefore$ How many subsets have sum = 4.

$dp[j]$ = no. of ways to get sum j.

what sum did you need before adding it (num)?

$$= j - \text{num}$$

because

$$\text{previous sum} + \text{num} = j$$

so $dp[j - \text{num}]$ = how many way i can make remain part

ex: nums = [1, 2] ; sum = 3.

step 1: decide to take 2

if i take 2 then i still need

$$3 - 2 = 1$$


```
def TargetSumWays(nums, target):
```

```
    total = sum(nums)
```

```
    if (total + target) % 2 != 0 or abs(target) > total:
```

```
        return 0
```

```
    subsetSum = (total + target) // 2
```

```
    dp = [0] * (subsetSum + 1)
```

```
    dp[0] = 1
```

```
    for num in nums:
```

```
        for j in range(subsetSum, num - 1, -1):
```

```
            dp[j] += dp[j - num]
```

```
    return dp[subsetSum]
```

2. Partition Equal Subset Sum

can we split array into 2 subsets with equal sum.

If total = sum(nums)

we need subset1 = subset2 ; subset1 + subset2 = total

subset1 = total / 2

∴ Finally problem is

Can we find subset with sum $\boxed{\text{target} = \text{total} // 2}$

if total is odd $\rightarrow$ its impossible.

```
def CanPartition(nums):
```

```
    total = sum(nums)
```

```
    if total % 2 != 0: return False
```

```
    target = total // 2
```

```
    dp = [False] * (target + 1)
```

```
    dp[0] = True
```

```
    for num in nums:
```

```
        for j in range(target, num - 1, -1):
```

```
            dp[j] =
```

```
                dp[j] or dp[j - num]
```

```
    return dp[target]
```


3. Palindrome Partitioning Backtracking + DP.

$s = "aab"$ o/p $\rightarrow [["a", "a", "b"], ["aa", "b"]]$

path = []

res = []

path.append("a")

res.append(path)

path.append("b")

res.append(path)

print(res)

// $[["a", "b"], ["a", "b"]]$ X

path.append('a')

res.append(path[:]) # copy

path.append("b")

res.append(path[:])

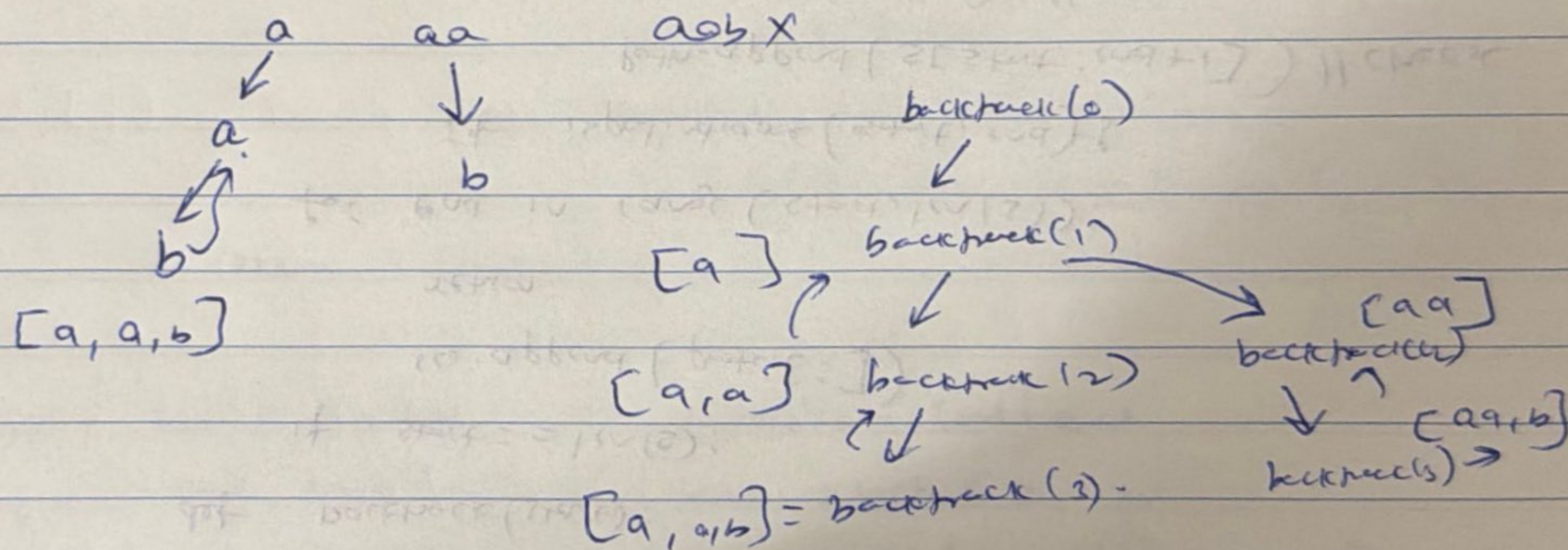
print(res)

$[["a"], ["a", "b"]]$ ✓

path is a reference (same object in memory)

path[:] creates new list with same values.

Start $\rightarrow$, try $\rightarrow$ a, aa, aab




```
def partition(s: str) →
```

```
    res = []
```

```
    path = []
```

```
    def backtrack(start):
```

```
        if start == len(s):
```

```
            res.append(path[:])
```

```
            return
```

```
        for end in range(start, len(s)):
```

```
            if ispalindrome(start, end):
```

```
                path.append(s[start:end+1]) // choose
```

```
                backtrack(end+1) // explore
```

```
                path.pop() // backtrack
```

```
    backtrack(0)
```

```
    return res
```

4. Palindrome Partitioning II

Return min no. of cuts needed for Palindrome Partitioning of 'S'.

S = "aab" o/p = 1

The Palindrome partitioning [aa, b] could be produced using 1 cut.

S = "abc" o/p = 2 a | b | c

isPal:

	0	1	2
0	T	T	F
1	F	T	F
2	F	F	T

dp:

	0	1	2
0	0	0	1

0 1 2

j = 1

dp[2] = min(dp[2], dp[2-1] + 1)


```
def minCut(s):
```

```
    n = len(s)
```

```
    ispal = [[False] * n for _ in range(n)]
```

```
    // precompute palindrome table
```

```
    for end in range(n):
```

```
        for start in range(end+1):
```

```
            if s[start] == s[end] and
```

```
                (end - start <= 2 or ispal[start+1][end-1]):
```

```
                ispal[start][end] = True
```

```
    // dp array
```

```
    dp = [0] * n    // min no of cuts for substring s[0:i]
```

```
    for i in range(n):
```

```
        dp[i] = i    // worst case: cut before every char
```

```
        for j in range(i+1):
```

```
            if ispal[j][i]:
```

```
                if j == 0:    // whole substring is palindrome
```

```
                    dp[i] = 0
```

```
            else:    // cut before j
```

```
                dp[i] = min(dp[i], dp[j-1] + 1)
```

```
    return dp[n-1]
```

1. first & last char match

$s[start] == s[end]$ and

2. either small string (s)

$end - start \leq 2$ or

or inner substring is palindrome $ispal[start+1][end-1]$